

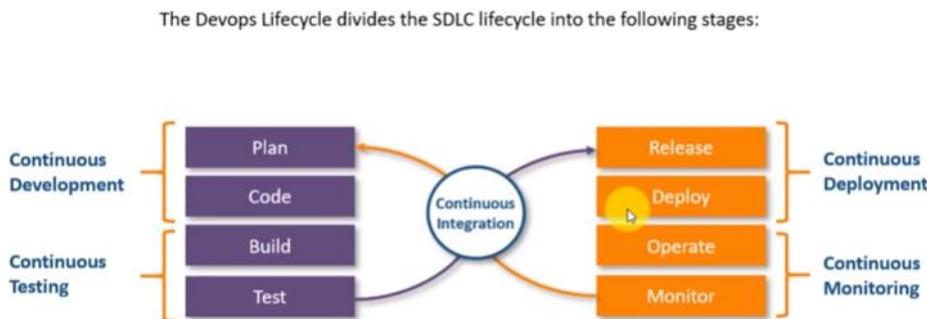
VIVA Questions for DevOps

1. What is DevOps?

Answer:

DevOps is a set of practices that combines software development (Dev) and IT operations (Ops). It aims to shorten the software development life cycle and deliver high-quality software continuously using automation, collaboration, and monitoring.

Explain DevOps Life cycle stages (Please refer DevOps ppt from Teams) along with tools.



2. How is DevOps different from Agile?

Answer:

Agile focuses on iterative development and collaboration between developers and customers. DevOps extends Agile by including operations teams, emphasizing automation, continuous integration/deployment, and faster delivery to production.

3. What are some popular DevOps tools?

Answer:

- **Version Control:** Git, GitHub, GitLab
- **CI/CD:** Jenkins, GitLab CI, CircleCI
- **Configuration Management:** Ansible, Puppet, Chef
- **Containers:** Docker

- **Container Orchestration:** Kubernetes
- **Monitoring:** Prometheus, Grafana, Nagios
- **IaC:** Terraform, CloudFormation

4. What is CI/CD?

Answer:

CI (Continuous Integration) is the practice of frequently integrating code changes into a shared repository.

CD (Continuous Deployment/Delivery) automates testing and deployment, ensuring that software can be released safely at any time.

5. What is the role of Jenkins in DevOps?

Answer:

Jenkins is an open-source CI/CD tool that automates building, testing, and deploying applications. It supports plugins and can integrate with many tools like Git, Docker, Maven, etc.

6. What is Docker and how is it different from a VM?

Answer:

Docker is a platform for developing, shipping, and running applications in containers. Containers share the host OS kernel and are more lightweight compared to VMs, which include the entire OS.

7. What is Infrastructure as Code (IaC)?

Answer:

IaC means managing and provisioning infrastructure through code instead of manual processes. Tools like Terraform and Ansible help in automating this process.

8. What is Git and how is it used in DevOps?

Answer:

Git is a distributed version control system that tracks changes in code. It allows teams to collaborate on code, manage versions, and integrate with CI/CD tools.

9. Explain the difference between `git fetch` and `git pull`.

Answer:

- `git fetch`: Downloads changes from the remote repository but doesn't merge them.
- `git pull`: Fetches and merges changes from the remote branch into the current branch.

10. What is the importance of monitoring in DevOps?

Answer:

Monitoring helps detect issues, track performance, and ensure system reliability. It supports proactive maintenance and better decision-making.

11. What is DevSecOps?

Answer:

DevSecOps integrates security practices into the DevOps process. It ensures security is addressed from the beginning of development, not as an afterthought.

12. What are branching strategies in Git?

Answer:

- **Feature Branching:** Separate branches for features.
- **GitFlow:** Uses branches for features, releases, hotfixes.
- **Trunk-Based Development:** Developers commit to a single branch (main/master) frequently.

13. How do Docker and Kubernetes work together?

Answer:

Docker packages applications in containers, and Kubernetes manages these containers at scale. Kubernetes schedules, deploys, and scales Docker containers efficiently.

14. What is the command to check the installed Docker version?

```
docker --version
```

15. How do you list all running containers?

```
docker ps
```

16 How do you list all containers, including stopped ones?

```
docker ps -a
```

17. How do you start a stopped Docker container?

```
docker start <container_id_or_name>
```

18. How do you stop a running container?

```
docker stop <container_id_or_name>
```

19. What command is used to remove a Docker container?

```
docker rm <container_id_or_name>
```

20. How do you remove all stopped containers?

```
docker container prune
```

21. How do you list all Docker images on your system?

```
docker images
```

22. How do you pull an image from Docker Hub?

```
docker pull <image_name>
```

23. What is Jenkins?

- Jenkins is an open-source automation server used for Continuous Integration and Continuous Deployment (CI/CD). It helps automate the building, testing, and deployment of applications.

24. What is Continuous Integration (CI)?

- CI is the practice of automatically integrating code changes from multiple contributors into a shared repository several times a day. Jenkins helps run automated builds and tests to validate code.

25. What is Continuous Deployment (CD)?

- CD refers to automatically deploying every successful code change to production. Jenkins automates this with pipeline scripts and plugins.

26. What are some key features of Jenkins?

- Key features:
 - Open-source and extensible
 - Supports distributed builds
 - Integration with version control systems
 - Plugin-based architecture
 - Support for pipelines as code (Jenkinsfile)

27. List CI/CD pipeline tools languages

CI/CD Tool	Pipeline Language
Jenkins	Groovy (Jenkinsfile)
GitLab CI/CD	YAML

28. What is Git?

Git is a distributed version control system used to track changes in source code during software development. It allows multiple developers to work on a project simultaneously.

29. How do you check the installed Git version?

```
git --version
```

30. How do you create a new Git repository?

```
git init
```

31. How do you clone an existing repository?

```
git clone <repository_url>
```

32. How do you check the current status of your Git repo?

```
git status
```

33. How do you add files to staging area?

```
git add <filename>      # Add a specific file  
git add .               # Add all files
```

34. How do you commit changes?

```
git commit -m "Your commit message"
```

35. How do you see the commit history?

```
git log
```

36. How do you check changes made to files?

```
git diff
```

37. How do you create a new branch?

```
git branch <branch_name>
```

38. How do you switch to a different branch?

```
git checkout <branch_name>
```

39. How do you create and switch to a new branch at once?

```
git checkout -b <branch_name>
```

40. How do you merge two branches?

```
git checkout main  
git merge <branch_to_merge>
```

41. How do you delete a branch?

```
git branch -d <branch_name>
```

42. How do you push changes to a remote repository?

```
git push origin <branch_name>
```

43. How do you pull changes from a remote repository?

```
git pull
```

44. What is the difference between git fetch and git pull?

- git fetch: downloads changes but does not merge.
- git pull: fetches and merges changes into your current branch.

45. How do you check remote repositories linked to your repo?

```
git remote -v
```

46. What are Ad Hoc Ansible commands?

Answer:

Ad Hoc Ansible commands are **one-liner** commands used to **perform quick tasks** on managed nodes **without writing a playbook**. They are useful for simple operations like checking connectivity, restarting services, or copying files.

47. What is the basic syntax of an Ansible Ad Hoc command?

Answer:

```
ansible <host-pattern> -m <module-name> -a "<module-arguments>"
```

Explanation of components:

- `ansible`: The Ansible command-line tool.
- `<host-pattern>`: Target hosts or group from the inventory (e.g., `all`, `webserver`s).
- `-m`: Specifies the **module** to use (e.g., `ping`, `shell`, `copy`).
- `-a`: Passes the **arguments** to the module (in quotes).

Example:

```
ansible all -m shell -a "uptime"
```

This runs the `uptime` command on all hosts defined in your inventory.

48. What is an Ansible Playbook?

An **Ansible Playbook** is a **YAML file** used to define a **set of tasks** that Ansible should run on managed hosts. It is the core way to **automate configuration, deployment, orchestration, and management** tasks.

Unlike Ad Hoc commands (used for one-off tasks), **playbooks are reusable, structured, and idempotent** — making them ideal for long-term automation.

49. Key Features of a Playbook

- Written in **YAML format**.
- Defines one or more **plays**, each targeting specific hosts.
- Contains **tasks** executed in order.
- Supports **variables, conditionals, loops, and roles**.
- Designed to be **idempotent** — running it multiple times results in the same system state.